

04 — Language Detection

What it is

Language detection is the task of automatically determining what language a piece of text is written in. In our project, we use a Python library called `langdetect` (based on Google's language-detection algorithm) to classify text as either Hungarian ("`hu`") or English ("`en`").

The algorithm works by looking at the statistical patterns of character combinations (n-grams) in the text and comparing them against known language profiles. Hungarian has very distinctive character patterns — letters like á, é, ő, ü, ■, ■ and common endings like "-nak", "-nek", "-ban", "-ben" make it quite easy to distinguish from English.

Real-life analogy

Imagine you're a mail sorter at an international post office. You don't read every letter — you just glance at the first few lines and look for clues. If you see words with lots of accented characters (á, é, ő, ü) and long compound words, it's probably Hungarian. If you see "the", "and", "is", it's English. You don't need to understand the content — the *shape* of the writing tells you the language.

That's exactly what `langdetect` does, except instead of a human glancing at text, it counts character patterns (like how often "th" appears vs "sz") and compares them against a statistical profile of each language.

Why we used it here

- **R4** requires automatic source-language detection with >95% accuracy.
- We need to know the source language for each record because:
 1. **Translation direction** (R3): If a category description is in Hungarian, we translate it to English (and vice versa). Without detection, we wouldn't know which direction to translate.
 2. **BM25 routing** (R9): The lemmatizer uses different models for each language — `huspacy` for Hungarian, `spaCy` for English. Sending Hungarian text to the English lemmatizer would produce garbage tokens.
 3. **Search API routing** (R19): The search API routes BM25 queries to `bm25_tokens_hu` or `bm25_tokens_en` based on the detected query language.

Default to Hungarian: Since the platform (`tablazat.hu`) is Hungarian-first and the vast majority of source data is in Hungarian, we default to "`hu`" when detection fails or the text is too short to classify reliably. This is a design decision, not a library default — better to occasionally mis-route an English record through Hungarian processing (where translation will catch it) than to send Hungarian text to the English pipeline (where it would be silently mangled).

Code walkthrough

```
from langdetect import detect, DetectorFactory

# langdetect uses randomization internally, which means the same text
# can return different results on different runs. Setting a seed makes
# it deterministic – same input always gives same output.
# This matters for our hash-based change detection (R10): if language
# detection flipped randomly, records would appear "changed" even when
# the source data hadn't changed, causing unnecessary re-processing.
DetectorFactory.seed = 0

def detect_language(text: str) -> str:
    """Detect whether text is Hungarian or English.

    Returns 'hu' or 'en'. Defaults to 'hu' for empty/undetectable text
    because the source platform (tablazat.hu) is Hungarian-first.
    """
    if not text or not text.strip():
        return "hu"        # No text to analyze → assume Hungarian

    try:
        detected = detect(text)    # Returns ISO 639-1 codes like 'hu', 'en', 'de'
    except Exception:
        return "hu"            # langdetect throws on very short or garbled text

    if detected == "hu":
        return "hu"
    if detected in ("en", "en-US", "en-GB"):
        return "en"            # Normalize English variants to just 'en'

    # Any other language (German, French, etc.) – shouldn't happen in our
    # dataset, but default to Hungarian since that's the platform language.
    return "hu"

def detect_record_language(record: dict) -> str:
    """Detect language for a full category or company record.

    Combines multiple text fields for better detection accuracy.
    A single field might be too short (e.g. company_name = "ABC Kft.")
    to detect reliably. By concatenating description + name + activities,
    we give the algorithm more text to work with.
    """
    text_parts = []

    # Try the most informative fields first
    for field in ("description", "short_description", "category_name", "company_name"):
        val = record.get(field)
        if val and isinstance(val, str) and val.strip():
            text_parts.append(val.strip())

    # If no text fields found, try activities list as fallback
    if not text_parts:
        activities = record.get("activities")
        if isinstance(activities, list):
            text_parts.extend(str(a) for a in activities[:5] if a)

    combined = " ".join(text_parts)
    return detect_language(combined)
```

How to explain this in an interview / to a teammate

"We use the `langdetect` library to automatically classify each record as Hungarian or English at the start of the pipeline. This drives three downstream decisions: which direction to translate, which NLP model to use for lemmatization, and which BM25 index column to write to. We set a deterministic seed so detection is repeatable across runs — important because our hash-based change detection would otherwise flag records as 'changed' when only the randomized detection result differed. We default to Hungarian when detection is uncertain because the source platform is Hungarian-first, and we combine multiple text fields from each record to give the algorithm enough text for reliable classification."